Technical Log Book

Edge AI Intrusion Detection System & Low-Level Packet Inspection

Project record: implementation, testing, results, limitations and final critical evaluation.

Phase 0: Conceptualization, Project Architecture & Environment Provisioning

Date of Execution: 24/06/2026

0.1 Project Ideation & Brainstorming Trajectory

The inspiration for this research stems from a first-hand observation of a network disruption exploit. An early encounter with a localized Distributed Denial of Service (DDoS) flood during a peer-to-peer gaming session resulted in a forced state disconnection, demonstrating how malicious traffic can exploit system vulnerabilities. This experience drove an interest in engineering proactive, low-overhead network defense mechanisms.

When initially brainstorming concepts for this project, the primary objective was to construct a dedicated secondary PC to test cybersecurity mitigation strategies and attack vectors. However, after further technical evaluation and resource assessment, the design was adapted to deploy a spare laptop as the target node. This transition reduced hardware procurement costs without compromising processing performance, link testing, or experimental validity.

In conjunction with this deployment, a two-node physical network topology was structured, consistent with the attacker/defender setup deployed in Phase 1. Coursework logic and network visualization tools were leveraged to model and monitor packet flow dynamics across the nodes in real time.

The architectural core of this defender focuses strictly on stateless network features rather than Deep Packet Inspection (DPI). DPI introduces substantial CPU overhead and presents data privacy concerns. The intended approach was to classify traffic using packet-header metadata without inspecting application payloads.

Original hypothesis: A machine-learning-driven network defender can identify and mitigate simulated volumetric attack traffic using low-level packet metadata while maintaining a low processing overhead on a dedicated edge device.

Note on Scope: The completed prototype provided packet capture, classification, and alerting, but no attack-blocking or rate-limiting functionality. The original mitigation objective was not achieved.

0.2 Agile Project Management Framework (Trello Architecture)

To maintain structured execution and prevent scope creep, project deliverables were organized using a pipelined Kanban framework via Trello. A distinct color-coded tracking scheme was established to categorize tasks across structural domains, including physical hardware provisioning, environment configuration, Python packet-parsing scripts, and machine learning pipeline integration.

Phase 1: Environment Provisioning, Bare-Metal OS Installation & Topology Verification

Date of Execution: 01/07/2026

1.1 Host Hardware Specifications (The Sandbox Endpoint)

Testing was conducted on a bare-metal Linux installation rather than in a virtual machine. A secondary host laptop was designated as the dedicated AI Defender node. Storage provisioning was managed using the Windows Disk Management utility to partition and prepare the boot media.

Proprietary Drivers: Third-party proprietary packages for graphics and Wi-Fi hardware were installed during setup to prevent CPU cycle waste on software-emulated rendering.

Storage Optimization: Full-disk cryptographic encryption was intentionally omitted to prevent storage I/O latency noise from interfering with downstream machine learning model performance evaluations.

1.2 Operating System Environment & Provisioning Pipeline

To safeguard the local production network from disruption during malicious traffic generation, an isolated two-node physical topology was engineered via a direct point-to-point Ethernet link bridging the primary workstation (Attacker) and the Linux target (Defender). Because this direct link lacks a DHCP server, static IPv4 addresses were hardcoded across the Layer 2 network interfaces.

Node Role	Hardcoded IPv4 Address	Subnet Mask	Default Gateway
Main Workstation (Attacker)	192.168.99.1	255.255.255.0	None
Ubuntu Target (Defender)	192.168.99.2	255.255.255.0	None

1.3 Link-Layer Connectivity & Empirical Verification

Following interface flushes and static IP assignment on the Linux interface (enp3s0), physical and data-link layer integrity was validated via ICMP diagnostics executed from the Windows terminal (ping 192.168.99.2). Results confirmed 0.0% packet loss and round-trip latency consistently <1 ms (Average = 0 ms).

1.4 Technical Friction Points & Resolution Log

Issue: Asymmetric ICMP Filtering. Initial diagnostic pings initiated from the Ubuntu node to the Windows host resulted in 100% packet loss, whereas pinging from Windows host to Ubuntu yielded 0% loss. The Windows host firewall was automatically dropping unauthenticated ICMP Echo Requests originating from external subnets.

Resolution: A targeted firewall rule was configured via Command Prompt: netsh advfirewall firewall add rule name="Allow ICMPv4 Inbound" protocol=icmpv4:8,any dir=in action=allow.

1.5 Network Topology Optimization & Throughput Baselining

The wireless NIC on the Ubuntu node was disabled to reduce background network traffic during testing. Routing table conflicts with active virtual miniport adapters were resolved.

Hardware Buffer Constraints: Analysis revealed that the legacy Ethernet NIC on the Ubuntu node enforces a ring buffer limit of 256 RX descriptors. Under high traffic rates, slow user-space processing may contribute to receive-buffer pressure and packet drops. A 10-second transport-layer stress test was executed using iperf3 over port 5001. A sustained throughput of approximately 900 Mbps on the 1 Gbps Ethernet link was achieved, confirming link operational stability.

Phase 2: Low-Level Kernel Ingestion, Asynchronous Parsing Engine & Telemetry Serialization

Date of Execution: 18/07/2026

2.1 Low-Level Socket Binding & Kernel Space Ingestion

To eliminate wrapper latency from third-party libraries, the ingestion engine hooks directly into the Linux kernel network stack using raw sockets (socket.AF_PACKET, socket.SOCK_RAW, protocol ETH_P_ALL). This configuration forces the kernel to copy incoming link-layer frames straight into user-space memory buffers before native IP stack filtering.

2.2 Decoupled Multi-Threaded Architecture

The ingestion thread receives frames using the socket's blocking recvfrom() operation. Packet parsing and queue dispatch are kept separate from CSV writing, which is performed by a background consumer thread. This producer-consumer design reduces the amount of work performed before the next receive operation.

Ingestion Thread: Receives frames via blocking recvfrom(65535), filters destination IP (192.168.99.2), extracts stateless header fields via memory slicing, and pushes data arrays onto a thread-safe FIFO queue (queue.Queue).

Asynchronous I/O Daemon: Background consumer thread (async_writer_worker) polls the FIFO queue, executing disk writes, vector serialization, and sliding window calculations independently.

2.3 Stateless & Stateful Feature Extraction Engine

The ingestion layer decodes raw binary network arrays using Python's struct module with big-endian network byte ordering (!):

Layer 2 & Layer 3 Decoupling: Initial 14 bytes are unpacked to evaluate Ethernet EtherType (non-IPv4 != 0x0800 dropped). Base 20-byte IPv4 header is unpacked; bitwise mask (& 0x0F) retrieves IHL to determine Layer 4 transport offsets.

Layer 4 Transport Processing: For TCP frames (IP Proto 6), TCP flags are extracted using bitwise operations: SYN = (flags & 0x02) >> 1, ACK = (flags & 0x10) >> 4, RST = (flags & 0x04) >> 2.

Stateful Rolling Window Tracking: A thread-locked sliding window (collections.deque) tracks arrival timestamps within a strict 1.0-second slice ($\Delta t = 1.0s$) to derive localized packet rates.

2.4 CSV Vector Serialization Schema

Feature vectors are serialized into network_telemetry.csv following the schema:

```
Timestamp, Packet_Length, Proto_TCP, Proto_UDP, Proto_ICMP, TCP_SYN, TCP_ACK, TCP_RST, Packet_Rate_1s
1784392812.421042, 64, 1, 0, 0, 1, 0, 0, 14.0
1784392812.428115, 1500, 1, 0, 0, 0, 1, 0, 18.0
```

2.5 Technical Friction Points & Resolution Log

Console print() statements in early iterations introduced I/O terminal blocking, causing ring buffer overflow and dropped frames. Terminal output formatting was removed from the core capture loop, reducing the observed frame drop issues during testing.

Phase 3: Adversarial Data Engineering, Preprocessing Pipeline & ML Model Optimization

Date of Execution: 04/08/2026

3.1 Adversarial Simulation & Telemetry Dataset Compilation

Telemetry data was compiled across two distinct environments:

Benign Traffic Generation (N = 26,491): High-throughput network activity via iperf3 (500 Mbps TCP stream) plus HTTP/DNS requests (variable payloads 40 B–65,380 B; 99.96% ACK prevalence).

Malicious SYN Flood Simulation (N = 3,083): Adversarial traffic via nping targeting 192.168.99.2:5001 at 1,000 pkts/sec (zero payload, 40 B headers, 100% TCP_SYN, 0% ACK).

Metric	Benign Traffic	Malicious SYN Flood
Sample Count (N)	26,491	3,083
TCP SYN Ratio	0.04% (11 packets)	100.00% (3,083 packets)
TCP ACK Ratio	99.96% (26,480 packets)	0.00% (0 packets)
Packet Length Range	40 B – 65,380 B	Fixed 40 B

3.2 Data Preprocessing Engine

Raw CSV logs were processed via prep_dataset.py to assemble normalized feature matrices:

Ground-Truth Labeling & Merging: Assigned binary target labels (y = 0 benign, y = 1 malicious) across 29,574 instances.

Feature Selection: Dropped Timestamp to prevent time-of-day overfitting, restricting features to stateless attributes and rolling velocity metrics.

Stratified Train-Test Split: Model performance was evaluated using the stratified 20% hold-out test subset created during preprocessing (N = 5,915). The remaining 80% (N = 23,659) was used for training. Stratification preserved the class proportions (89.58% benign / 10.42% malicious) in both subsets. This was a single train-test split, not Stratified K-Fold cross-validation.

Z-Score Standardization: Continuous features (Packet_Length, Packet_Rate_1s) were standardized using StandardScaler fitted only on training data (transforming continuous features to approximately zero mean and unit variance on the training subset).

Phase 4: Offline Model Training & Evaluation

Date of Execution: 04/08/2026

4.1 Quantitative Evaluation Metrics

A Random Forest Classifier was trained on preprocessed matrices. Evaluated on the held-out test subset (N = 5,915), the classifier recorded the following metrics:

Metric	Score	Analytical Significance
Accuracy	100.00%	Total correct predictions across test subset
Precision	100.00%	Zero false positives (FP = 0) within test subset
Recall	100.00%	Zero false negatives (FN = 0) within test subset
F1-Score	100.00%	Harmonic mean of precision and recall
ROC-AUC	1.0000	Separability across decision thresholds on test subset

Critical evaluation of offline results: The Random Forest achieved 100% on the held-out test subset from this dataset. However, this result reflects separation within a controlled and relatively narrow dataset, not proven generalisation to different networks or attack behaviours. The benign and malicious samples had highly distinctive characteristics: the simulated SYN flood consisted of fixed-length TCP packets with SYN asserted and ACK unset, whereas the benign capture had a very different flag distribution and packet-length

profile. The random stratified split also allowed samples from the same capture conditions to appear in both training and testing subsets. Consequently, the test set was unseen at row level but was not an independent capture-session evaluation. The perfect scores therefore exposed a limitation in the experimental dataset: it was too easy for the model to separate the two collected classes. Further training and testing on varied, independently captured benign traffic and multiple attack types would be necessary before claiming robust detection.

Phase 5: Real-Time Engine Deployment & Live Detection

Date of Execution: 04/08/2026

5.1 Operational Infrastructure & Live Empirical Analysis

The live detection script (live_ids.py) binds to enp3s0 via AF_PACKET raw sockets. Live verification was conducted against low-rate SYN flood probes (192.168.99.1 targeting 192.168.99.2:5001 at 9–20 pps).

Under low-rate probing, raw Random Forest threat probabilities fell to 27.0%–28.0% (below the standard 50.0% decision threshold) because the velocity feature Packet_Rate_1s produced smaller Z-score transforms than observed during high-rate training (~1,000 pps).

5.2 Compound Decision Rule

To detect low-rate probes, a compound decision rule was introduced:

Decision(x) = 1 (Malicious) IF [P(y=1|x) >= 0.20 AND TCP_SYN == 1 AND TCP_ACK == 0] OR [P(y=1|x) >= 0.50]

Decision(x) = 0 (Benign) OTHERWISE

No false positives were observed in the benign traffic sample used for this live test. This result is limited to that sample and should not be interpreted as a general false-positive rate.

Critical evaluation of the live detection approach: The live tests showed that the raw Random Forest probability was not a reliable standalone decision threshold for the lower-rate probes used during deployment. The compound rule improved the number of simulated SYN probes that triggered alerts, but it also made the decision heavily dependent on a simple TCP flag condition: SYN = 1 and ACK = 0, combined with a probability threshold. This is close to a rule-based SYN/ACK check rather than a sufficiently discriminating machine-learning intrusion detector. An initial SYN packet can also occur during a legitimate TCP connection attempt, so treating this flag combination as sufficient evidence of an attack risks false positives in normal traffic. The limited benign test sample did not establish that this rule would maintain a low false-positive rate in a wider network environment. The rule therefore demonstrated a way to detect the specific probes used in testing, but it did not establish robust attack detection.

Phase 6: Real-Time Inference Latency Benchmarking & System Profiling

Date of Execution: 07/08/2026

6.1 Experimental Methodology & Benchmark Results

Timing measurements were collected via benchmark_ids.py over 1,000 benchmark iterations using time.perf_counter() to evaluate single-frame execution paths (scaling + feature vector creation + inference).

Metric	Measured Value	Operational Interpretation
Total Benchmark Iterations	1,000	Sample space for statistical distribution

Metric	Measured Value	Operational Interpretation
Mean Processing Time (μ)	24,598.86 μs (24.60 ms)	Mean measured feature preparation, scaling and inference time per frame
99th Percentile Latency (p99)	47,689.98 μs (47.69 ms)	99th-percentile latency in the measured runs
Theoretical Max Throughput	~41 packets/sec	Theoretical rate for the benchmarked processing path

6.2 Performance Bottleneck Analysis

Critical evaluation of benchmark performance: During the low-rate live tests (approximately 9–20 packets/sec), the engine processed the observed attack traffic without reported packet loss. The benchmark measured a mean single-frame processing time of approximately 24.60 ms, equivalent to a theoretical rate of around 41 frames/sec for that measured processing path. This is substantially below the packet rate required for high-throughput or line-rate capture. The result therefore identifies per-frame Python inference and feature preparation as a major performance limitation. The low-rate experiment demonstrates operation under the tested conditions only; it does not demonstrate the ability to sustain volumetric flood traffic.

Potential future optimisations include investigating ONNX inference, compiled packet-processing code, and alternative capture mechanisms such as eBPF or DPDK; these were not implemented or tested in this project.

Phase 7: Project Conclusion & Critical Reflection

The original objective was to develop a machine-learning-driven network defender capable of identifying and mitigating volumetric attacks while maintaining low processing overhead. The project achieved some of its technical objectives, but it did not achieve the mitigation objective and did not produce a robust, general-purpose intrusion detection system.

A working prototype was developed to capture Ethernet frames using Linux AF_PACKET raw sockets, parse selected IPv4 and TCP header fields, extract packet-level features, record telemetry, and apply a Random Forest classifier. The project also established a repeatable two-node test environment and measured the processing cost of the inference pipeline.

The offline model achieved 100% accuracy, precision, recall, and F1-score, with an ROC-AUC of 1.0000, on the stratified hold-out subset. Initially, these results appeared to indicate excellent classification. However, analysis of the dataset showed that the two classes were unusually easy to distinguish: the simulated malicious traffic consisted of highly consistent SYN-only packets, while the benign traffic had a markedly different distribution. The random split did not test generalisation to a separate capture session or a different network. The 100% result was therefore a property of the limited experimental dataset, not evidence that the model could reliably identify attacks in realistic, varied traffic.

Live testing exposed a further weakness. Low-rate SYN probes produced Random Forest probabilities of approximately 27–28%, below the original 50% decision threshold. A compound rule was introduced to trigger alerts for packets with SYN asserted and ACK unset when the model probability exceeded 20%. This improved detection of the particular probes tested, but it made the final decision logic heavily dependent on a simple TCP flag check. In practical terms, the final system behaved more like a simple SYN-without-ACK heuristic detector than a robust machine-learning NIDS. Legitimate TCP connection attempts can also begin with SYN set and ACK unset, so this rule could generate false positives in a wider network environment. The small benign test sample was not sufficient to rule this out.

The project also failed to meet its original mitigation aim. The system generated alerts and classified packets; it did not block, rate-limit, or otherwise stop malicious traffic. It should therefore be described as a prototype detection and monitoring system, not an attack mitigation system. This is an important

shortcoming against the original hypothesis, and the hypothesis was not fully supported.

Performance testing identified another limitation. The measured mean processing time was approximately 24.60 ms per frame, corresponding to a theoretical single-threaded processing rate of about 41 frames per second for the measured path. This may be adequate for the low-rate probes used in testing, but it is not sufficient to claim high-throughput or line-rate protection. The project did not demonstrate an ultra-low CPU footprint or sustained processing under a high-volume attack.

The main learning outcome was that successful model training and high offline scores do not guarantee a useful deployed security system. The live tests revealed weaknesses that were not apparent from the offline metrics, and the attempt to compensate with a heuristic rule exposed the danger of overfitting the detection logic to one attack signature. The project improved my understanding of raw packet capture, binary protocol parsing, concurrency, feature engineering, model evaluation, and the difference between a laboratory result and a deployable system.

Further work would require collecting a much broader dataset across independent capture sessions, including varied legitimate TCP connection attempts and multiple attack types. The model should be compared against a simple rule-based baseline using a separate test environment, with false positives and false negatives reported explicitly. The detection policy should use traffic context and rates over time rather than relying on a single SYN/ACK pattern. Finally, mitigation would need to be designed and tested as a separate component, with safeguards to avoid blocking legitimate traffic. Inference would also need substantial optimisation before the system could be considered for high-throughput deployment.

Overall conclusion: The project successfully produced and evaluated a low-level packet-capture and alerting prototype, but it failed to achieve its original goal of mitigating attacks and did not establish robust general-purpose detection. Recognising and explaining these limitations is a central outcome of the project.